Lecture 03.16: Types of Functions & Parameter Semantics

📌 Executive Summary

Built-in vs user-defined functions, positional vs keyword arguments, default parameter pitfalls, *args and **kwargs variadic parameters.

🔬 Theoretical Foundations & Principles

1. The Taxonomy of Parameters & Arguments

- **Positional Arguments:** Matched by position order.
- **Keyword Arguments:** Matched by parameter name identifier.
- **Default Arguments:** Fallback values evaluated once at function definition time.
- **Variadic Positional (`*args`):** Collects excess positional arguments into a `tuple` .
- **Variadic Keyword (`**kwargs`):** Collects excess keyword arguments into a `dict` .

2. The Mutable Default Parameter Trap

Default parameter values are evaluated once when the `def` statement executes, NOT upon each call! Never use mutable objects (`[]` , `{}`) as default arguments.

💻 Production Implementation & Code

```
# Variadic function demonstration
def flexible_logger(level: str, *messages: str, **metadata: str) -> None:
    combined = " ".join(messages)
    meta_str = " | ".join(f"{k}={v}" for k, v in metadata.items())
    print(f"[{level.upper()}] {combined} ({meta_str})")

flexible_logger("info", "Server started", "Listening on port 8080", env="productic
```

⚙️ Algorithmic Complexity & Systems Analysis

- **Time Complexity:** Detailed in respective algorithmic operations.
- **Auxiliary Space:** Memory bounds and CPython frame management.